

Контейнеризация



Нижний Новгород, 2025г.

Для чего нужен (или плюсы):

1. Изолированный запуск приложений в контейнерах.
2. Упрощение разработки, тестирования и деплоя приложений.
3. Отсутствие необходимости конфигурировать среду для запуска — она поставляется вместе с приложением — в контейнере.
4. Упрощает масштабируемость приложений и управление их работой с помощью систем оркестрации контейнеров.

Преимущества docker

Image
Management

Resource
Isolation

File system
Isolation

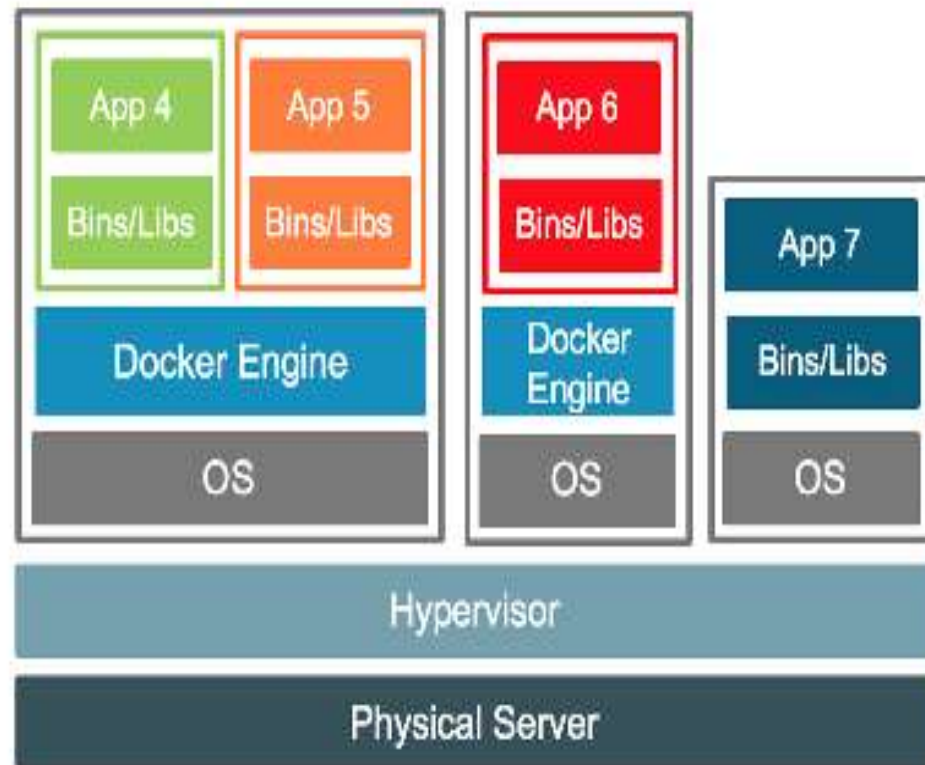
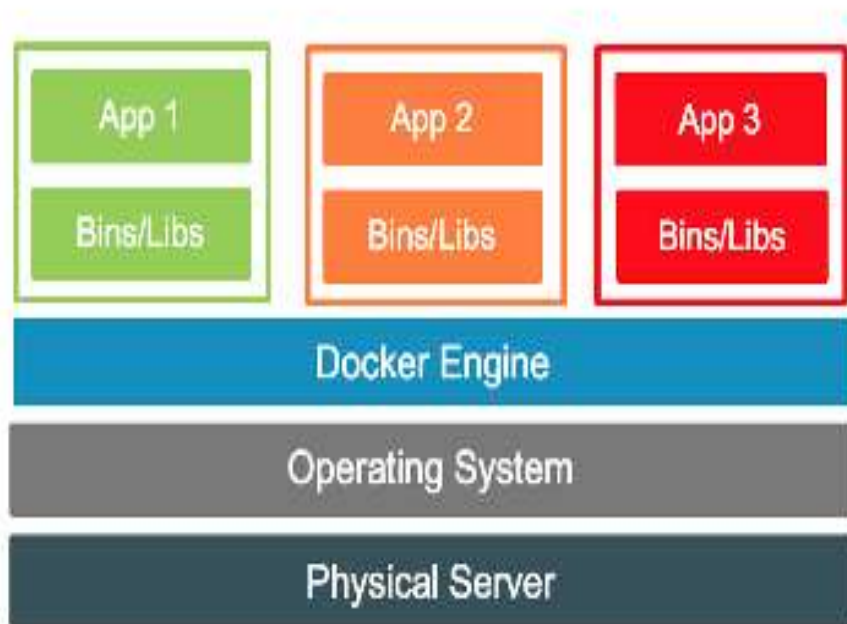
Network
Isolation

Change
Management

Sharing

Process
Management

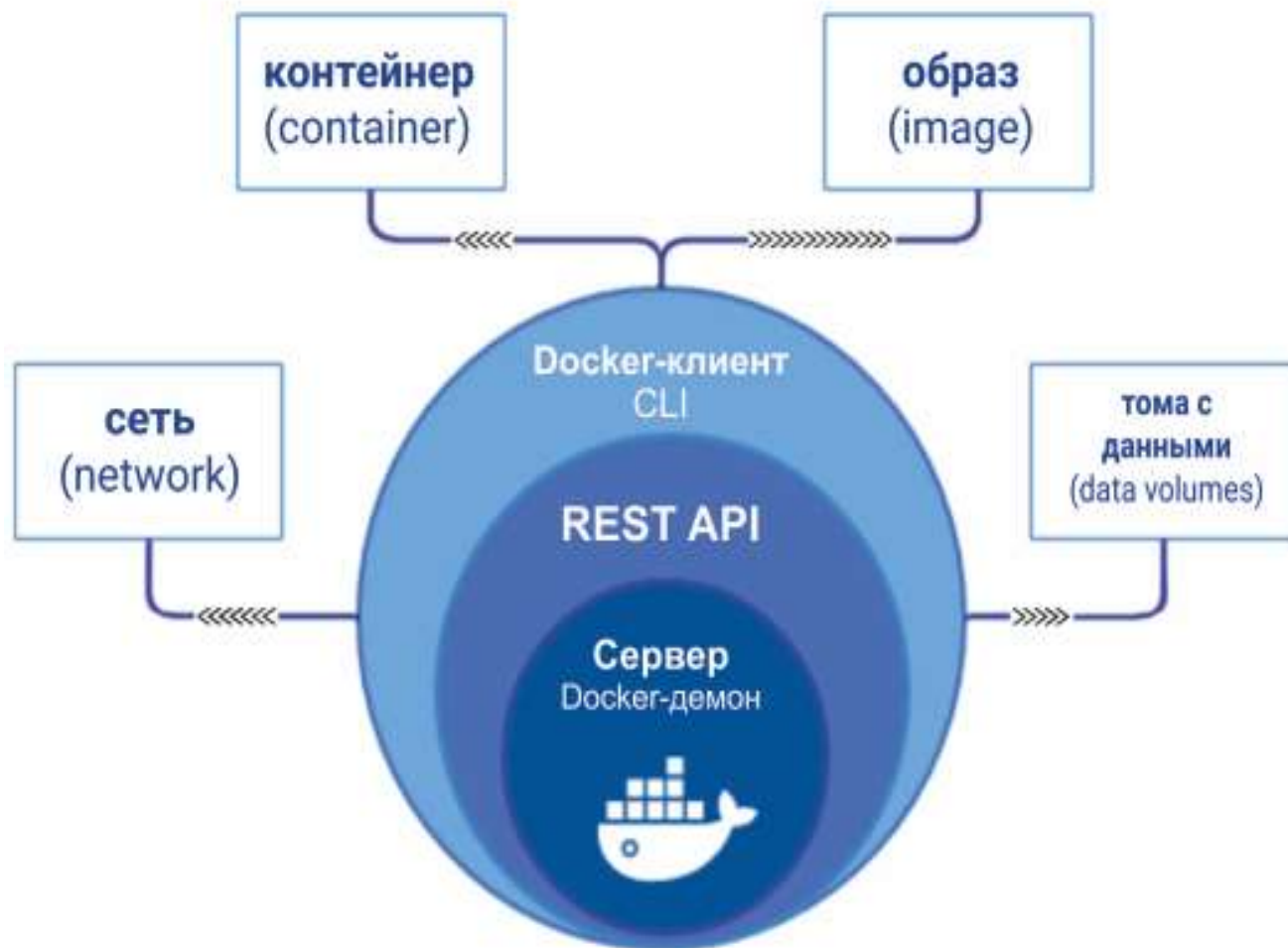
Чем отличается от Виртуальных Машин?



Из чего состоит?

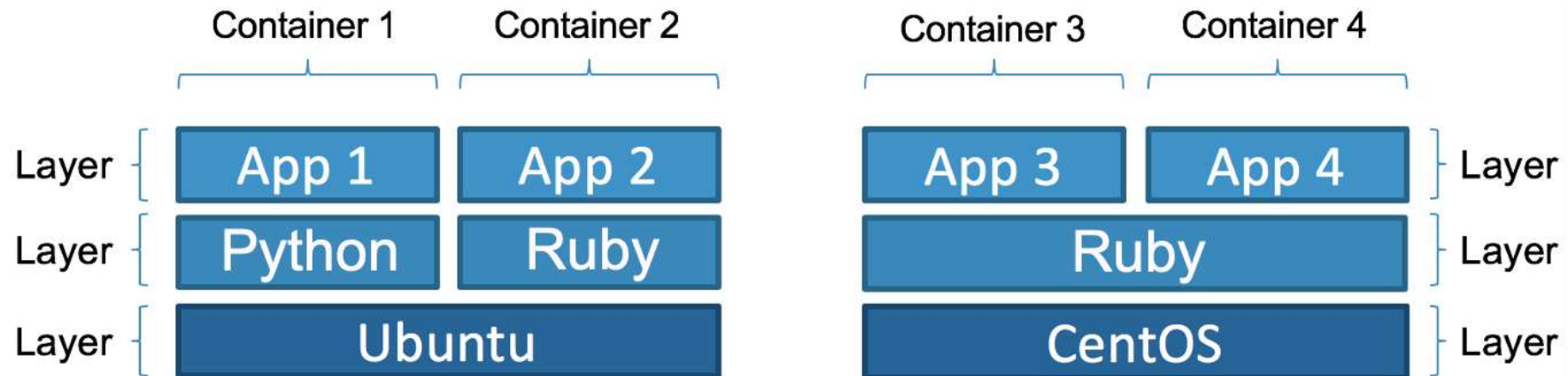
1. Образы
2. Контейнеры
3. Volumes
4. Networks

Компоненты Docker Engine



Образ

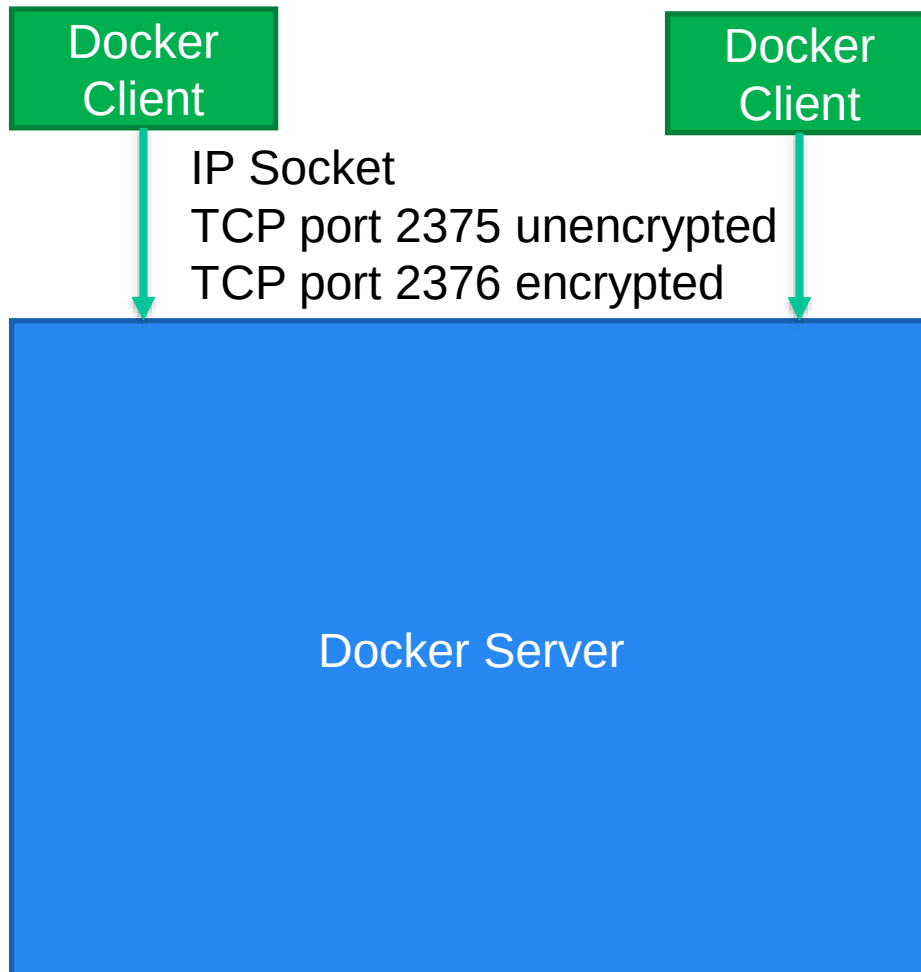
- Read-only шаблон с набором инструкций, предназначенных для создания контейнера
- Образ состоит из неизменяемых слоев, каждый из которых добавляет/удаляет/изменяет файлы из предыдущего слоя.
- Неизменяемость слоев позволяет их использовать совместно в разных образах.



Контейнеры

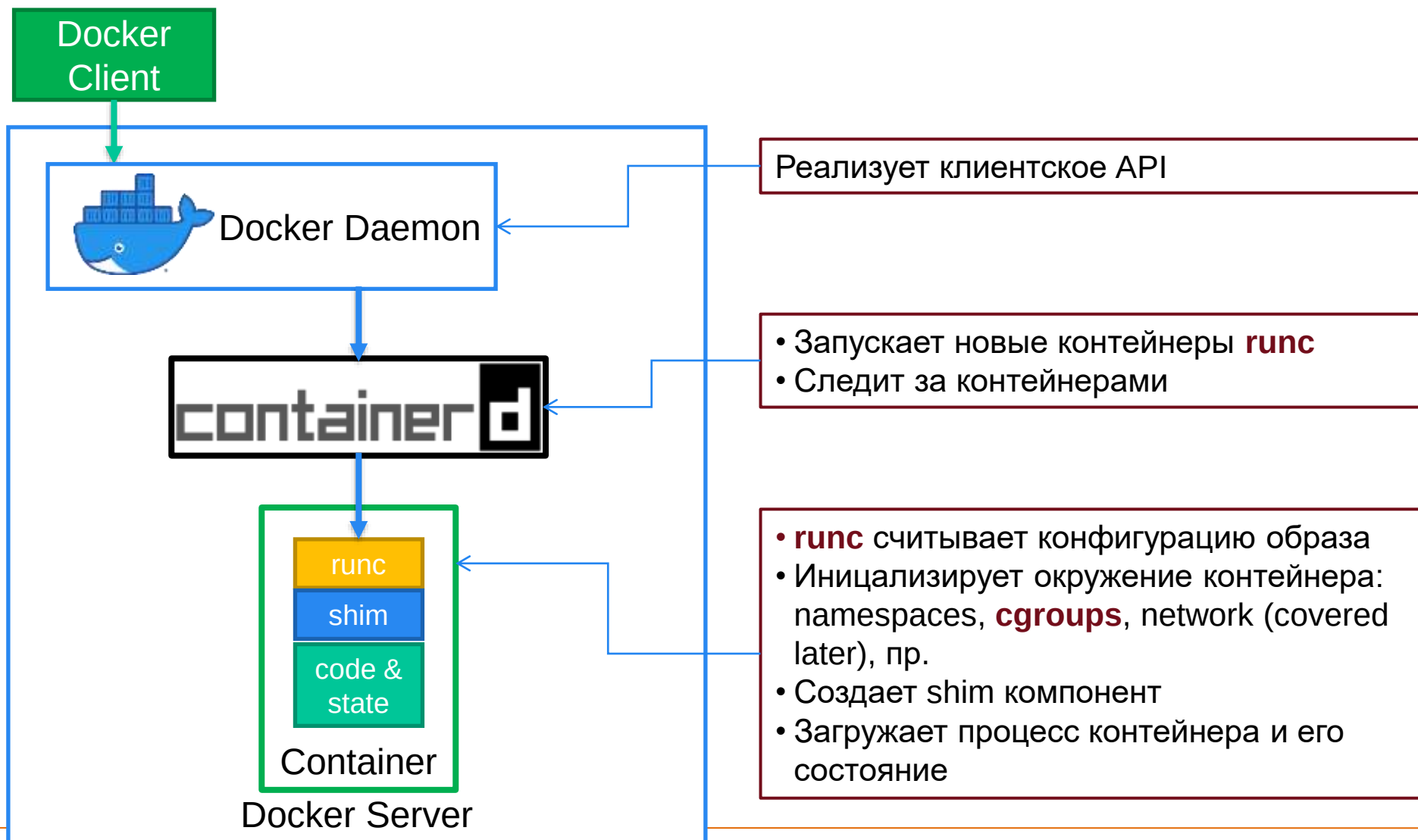
Контейнер - это запущенный образ.

Архитектура docker

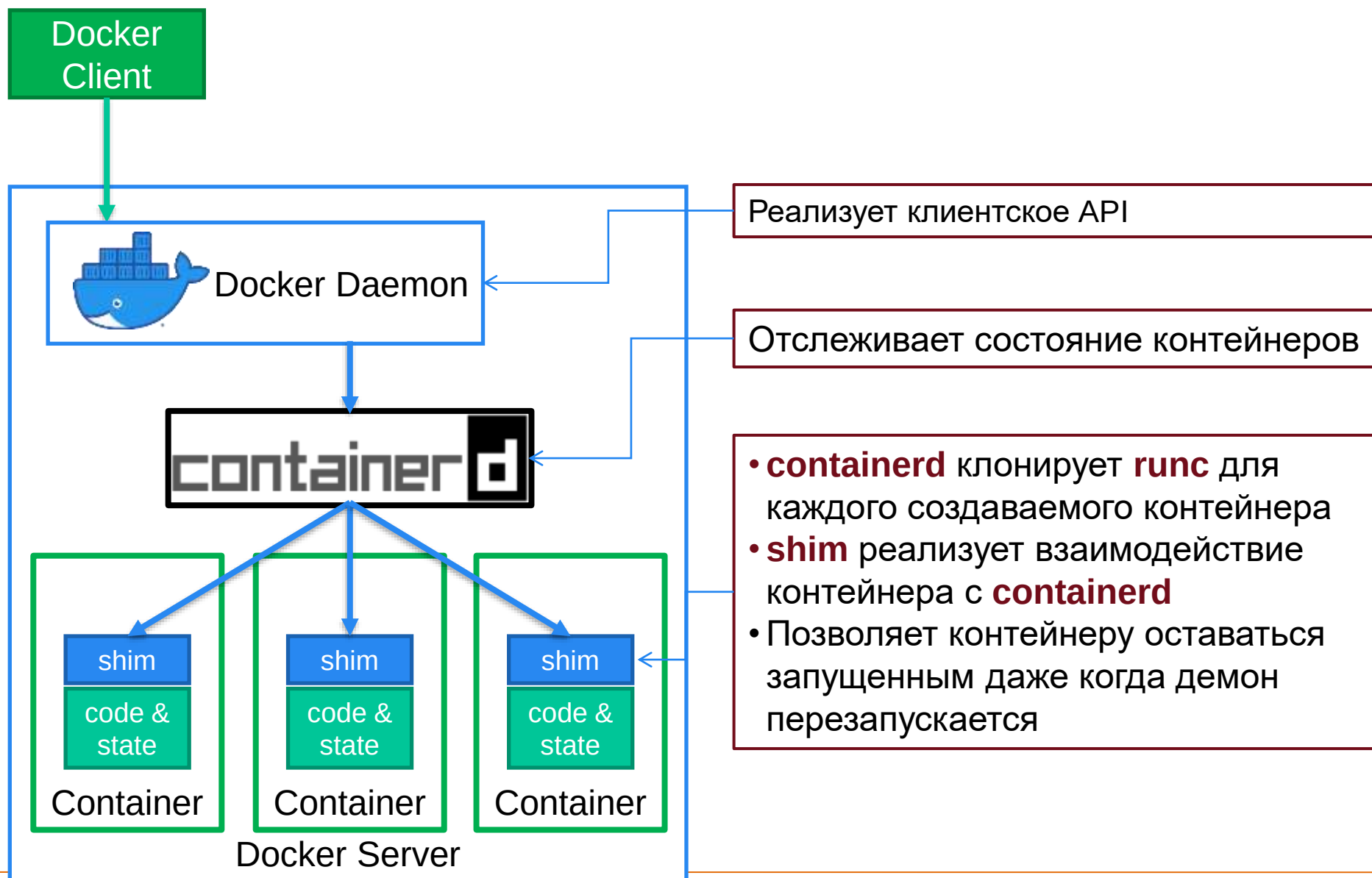


- Клиент-серверная архитектура
- Использует IP-сокеты для коммуникации
- Клиенты могут находиться на одном сервере и взаимодействовать через сеть

Архитектура docker

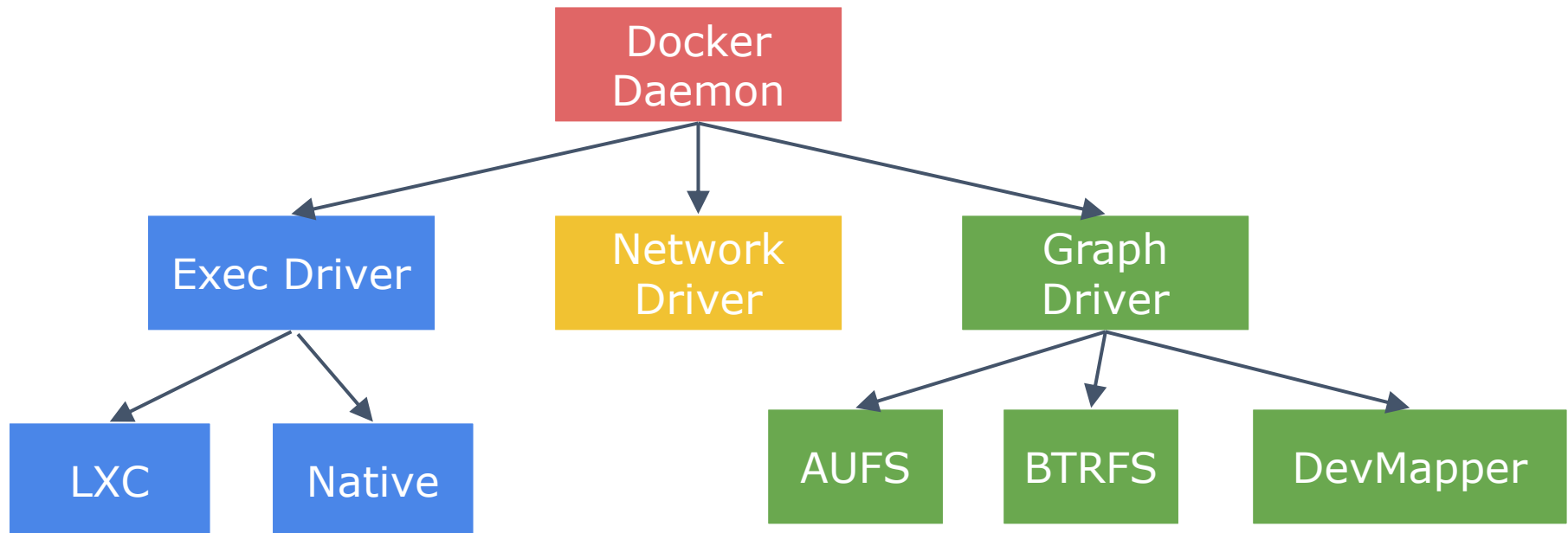


Архитектура docker



Структура гипервизора docker

Docker Daemon (dockerd) — это фоновый процесс, который управляет всей жизнью контейнеров на хосте: их созданием, запуском, остановкой и удалением. Для выполнения этих задач он использует несколько "драйверов" (модулей), каждый из которых отвечает за свою специфическую область.



Alpine Linux — это минималистичный дистрибутив Linux

- Использует легковесную библиотеку **musl** с вместо **glibc**.
- Имеет минимальный набор shell-утилит.
- Монтирует минимальный набор файлов и директорий для обеспечения взаимодействия с ядром хостовой системы Linux.
- Используется в качестве **базового производного слоя** для вашего образа.

Аспект	Объединяющие ФС (Union FS)	ФС со снимками (Snapshotting FS)	Блочные устройства с записью при изменении (Copy-on-write block devices)
Инициализация	Очень быстро / Очень дешево	Быстро / Дешево	Быстро / Дешево
Изменение мелких файлов	Очень быстро / Очень дешево	Быстро / Дешево	Быстро / Затратно
Изменение крупных файлов	Медленно (в первый раз) / Неэффективно (копирование при изменении!)	Быстро / Дешево	Быстро / Дешево
Сравнение изменений (Diffing)	Очень быстро	Очень быстро	Медленно
Использование памяти	Эффективно	Эффективно	Неэффективно (при высокой плотности)
Недостатки	Случайные странности, AUFS не в основной ветке, IAUFs имеет больше странностей	ZFS не в основной ветке, BTRFS не так хороша	Высокое использование диска, Отличная производительность (кроме сравнения)
Итог	Идеально для PaaS и задач с высокой плотностью	Это будущее (вероятно)	Мощное, но специализированное решение (как Dodge Ram 3500)

Что делает docker при запуске образа?

```
$ docker run --rm -p 8888:80 nginx
```

- **скачивает образ**
- **создает контейнер**
- **инициализирует файловую систему и монтирует read-only образ**
- **инициализирует сеть/мост**
- **запускает указанный процесс**
- **обрабатывает и выдает вывод приложения**

Установка docker

```
$ docker -v
```

```
Docker version 20.10.2, build 2291f61
```

```
# Скачиваем скрипт установки docker
```

```
curl -fsSL https://get.docker.com -o get-docker.sh
```

```
# запускаем скрипт
```

```
sudo sh get-docker.sh
```

```
# добавляем текущего пользователя в группу docker (чтобы  
использовать команды без sudo)
```

```
sudo usermod -aG docker $USER
```

Основные команды

docker ps

Отображает список запущенных контейнеров

```
$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
05ba9499f696	nginx	"nginx -g 'daemon of..."	About a minute ago	Up About a minute	0.0.0.0:8888->80/tcp	affectionate_bouman

```
$ docker ps -a – отображает список всех контейнеров в системе
```

```
docker run --rm -it -v `pwd`:/app ruby:2.7-alpine ash
```

Запускает контейнер с Alpine Linux и установленным ruby.

docker run --rm -it -v `pwd` :/app **ruby:2.7-alpine ash**

Запускает команду в контейнере из образа

\$ docker run IMAGE [COMMAND]

ruby:2.7-alpine – запускаемый образ

ash – команда для запуска консоли

Если команда не указана, запускается команда по умолчанию, которая указана в конфигурации образа.

docker run --rm -it -v `pwd` :/app **ruby:2.7-alpine ash**

При установке параметра **--rm** контейнер после остановки будет удален.

```
$ docker ps -a
```

```
# запомнить количество контейнеров
```

```
$ docker run --rm ruby:2.7-alpine
```

```
$ docker ps -a
```

```
# количество контейнеров не изменилось
```

```
$ docker run ruby:2.7-alpine
```

```
# появился остановленный контейнер со статусом «Exited  
(0) X seconds ago»
```

docker run --rm -it -v `pwd` :/app ruby:2.7-alpine ash

Комбинация параметров `i` и `t` позволяет запустить контейнер в интерактивном режиме, таким образом вам станет доступной консоль внутри контейнера.

```
$ docker run --rm ruby:2.7-alpine ash
```

```
$ docker run --rm -it ruby:2.7-alpine ash
```

```
docker run --rm -it -v `pwd` :/app ruby:2.7-alpine ash
```

Монтирует локальную папку (pwd - выводит путь к текущей директории) в папку /app внутри контейнера.

docker run: параметр -p

```
$ docker run --rm -p 8888:80 nginx
```

Связывает локальный порт хостовой машины с портом приложения в контейнере

```
$ docker run --rm nginx
```

```
# открыть в браузере localhost
```

```
$ docker run --rm -p 8888:80 nginx
```

```
# открыть в браузере localhost
```

```
# открыть в браузере localhost:8888
```

Welcome to nginx!

If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to nginx.org.
Commercial support is available at nginx.com.

Thank you for using nginx.

docker run: параметр -d

```
$ docker run --rm -d -p 8888:80 nginx
```

Запускает контейнер в фоновом режиме, STDOUT приложения не выводится

docker stop [container]

Останавливает работу контейнера

```
$ docker ps
```

```
$ docker stop 05ba9499f696
```

```
$ docker ps
```

```
# Отсутствует запущенный контейнер с ID 05ba9499f696
```

```
# http://localhost:8888 стал недоступен
```

docker images

Отображает список доступных локально образов

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
<none>	<none>	40873f17b0a3	18 minutes ago	969MB
hello_world_app	latest	357bac0cc7ad	7 hours ago	860MB
ruby	2.7-buster	4ab308e18398	15 hours ago	842MB
ruby	3.0-buster	4b9aa88721e2	15 hours ago	850MB
debian	buster-slim	5b395270ee77	29 hours ago	69.2MB
debian	buster	dc2eddc15825	29 hours ago	114MB
async_arch	latest	f6be8625a6f5	8 days ago	1.2GB
ruby	2.7-alpine	d880f078af31	3 weeks ago	51.9MB
nginx	latest	35c43ace9216	3 weeks ago	133MB

Создание файлов внутри docker

```
$ docker run --rm -it -v $(pwd):/app ruby:2.7-alpine ash  
$ cd app  
$ touch file.txt  
$ exit  
$ nano file.txt # или другой редактор  
# что-нибудь написать, попытаться сохранить
```

Решение проблемы

```
$ ls -l
```

```
-rw-r--r--      1      root    root    ...    file.txt
```

```
# исправить права конкретному файлу
```

```
$ sudo chown user file.txt
```

```
# исправить права всем файлам в директории
```

```
$ sudo chown user -R dir
```



Dockerfile

Инструкции для подготовки образа

Пример Dockerfile

```
 Dockerfile > ...  
1 FROM ruby:2.7-buster  
2  
3 RUN apt-get update  
4  
5 ENV SRC_PATH /app  
6 RUN mkdir -p $SRC_PATH  
7 WORKDIR $SRC_PATH  
8  
9 ADD . .  
10  
11 CMD ["ruby", "test.rb"]
```

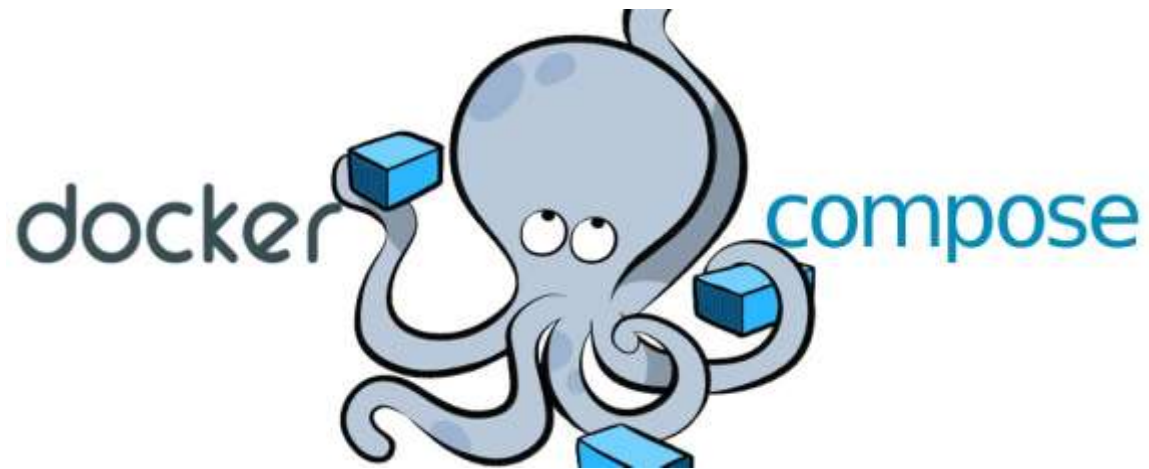
docker build . -t app_name

Сборка образа из Dockerfile

```
$ docker build . -t hello_world
```

```
→ hello_world git:(master) x docker build . -t hello_world
[+] Building 3.0s (10/10) FINISHED
  => [internal] load build definition from Dockerfile
  => => transferring dockerfile: 37B
  => [internal] load .dockerignore
  => => transferring context: 2B
  => [internal] load metadata for docker.io/library/ruby:2.7-buster
  => [internal] load build context
  => => transferring context: 1.90kB
  => [1/5] FROM docker.io/library/ruby:2.7-buster@sha256:2dac3a4b5fa750006eaf495a87da315ce7192032f5fc8b1d41e42631c4fcd7a0
  => => resolve docker.io/library/ruby:2.7-buster@sha256:2dac3a4b5fa750006eaf495a87da315ce7192032f5fc8b1d41e42631c4fcd7a0
  => CACHED [2/5] RUN apt-get update
  => CACHED [3/5] RUN mkdir -p /app
  => CACHED [4/5] WORKDIR /app
  => [5/5] ADD . .
  => exporting to image
  => => exporting layers
  => => writing image sha256:59e035ed121b5ffeea69ea8ee06743b60107d019c4542939daaf378cfcaf2407
  => => naming to docker.io/library/hello_world
```

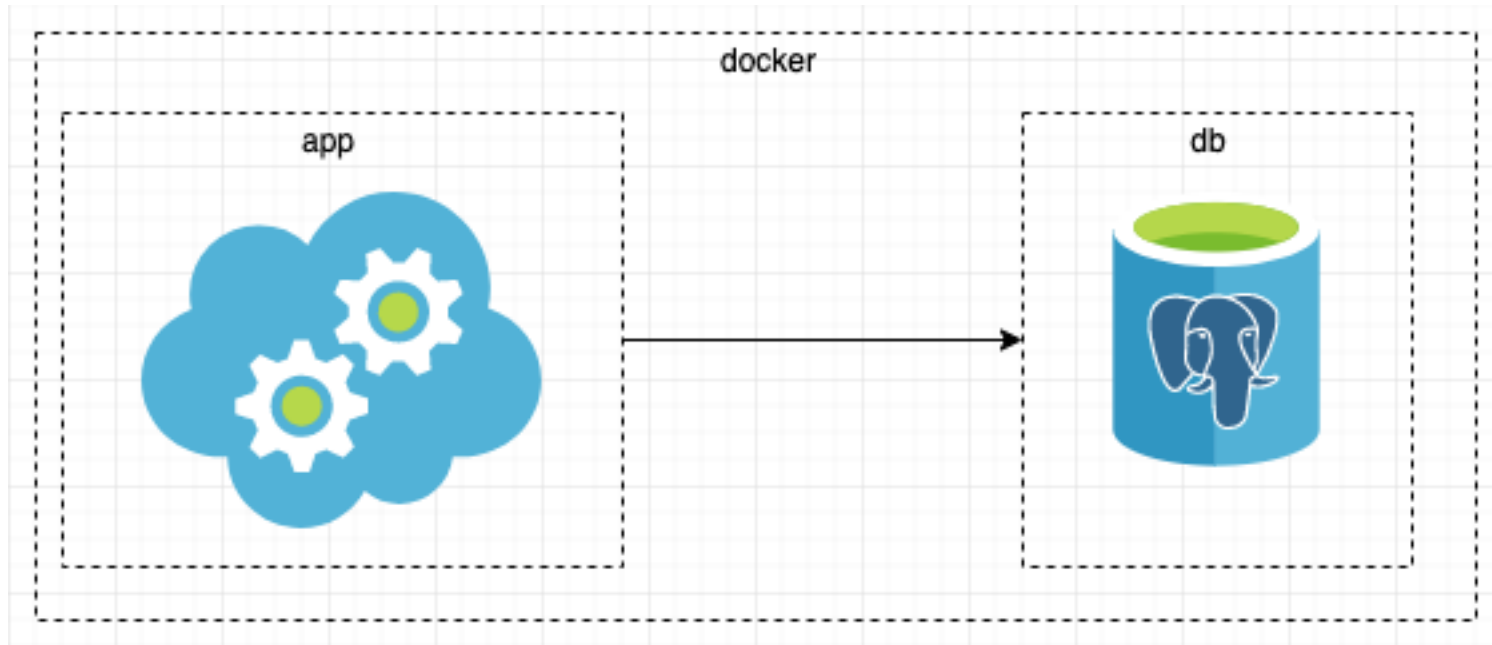
```
$ docker run --rm hello_world
```



docker-compose

- инструментальное средство, предназначенное для решения задач, связанных с развёртыванием проектов.

Пример приложения



Установка docker-compose (Linux)

<https://docs.docker.com/compose/install/>

Скачиваем docker-compose сразу в папку /usr/local/bin

```
$ sudo curl -L  
"https://github.com/docker/compose/releases/download/1.25.3/doc  
ker-compose-$(uname -s)-$(uname -m)" -o /usr/local/bin/docker-  
compose
```

Устанавливаем разрешение на запуск

```
$ sudo chmod +x /usr/local/bin/docker-compose
```

Проверяем

```
$ docker-compose --version
```

docker-compose.yml

```
🚢 docker-compose.yml > ...
1  services:
2    app:
3      build: .
4
5    nginx:
6      image: nginx
7      ports:
8        - '8888:80'
9      volumes:
10       - ./log:/var/log/nginx
11
12    curl:
13      image: ruby:2.7-buster
14      depends_on:
15        - nginx
16      working_dir: /app
17      volumes:
18        - ./curl:/app
19      command: curl -o result.log nginx
```

Основные команды: `up`

\$ `docker-compose up`

Создает все зависимости
(сеть, volumes) для запуска
сервисов, скачивает или
собирает образы и
запускает все сервисы
`docker-compose`

```
+ hello_world git:(master) x docker-compose up
Creating network "hello_world_default" with the default driver
WARNING: Found orphan containers (hello_world_curl_action_1) for this project. If you
mand with the --remove-orphans flag to clean it up.
Building app
Step 1/7 : FROM ruby:2.7-buster
--> 4ab308e18398
Step 2/7 : RUN apt-get update
--> Running in b862ee447624
Get:1 http://deb.debian.org/debian buster InRelease [122 kB]
Get:2 http://security.debian.org/debian-security buster/updates InRelease [65.4 kB]
```

```
Successfully built a5574b21857d
Successfully tagged hello_world_app:latest
WARNING: Image for service app was built because it did not already exist.
ld'.
Creating hello_world_nginx_1 ... done
Creating hello_world_app_1 ... done
Creating hello_world_curl_1 ... done
```

```
Attaching to hello_world_app_1, hello_world_nginx_1, hello_world_curl_1
app_1      | Hello world!
nginx_1    | /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
nginx_1    | /docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
nginx_1    | /docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
nginx_1    | 10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
nginx_1    | 10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
nginx_1    | /docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
curl_1     | % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
curl_1     |           % Dload  Upload    Total     Spent    Left     Speed
```

Основные команды: ps

\$ docker-compose ps

Выведет список запущенных контейнеров в рамках проекта.

```
→ hello_world git:(master) x docker-compose ps
```

Name	Command	State	Ports
hello_world_app_1	ruby test.rb	Exit 0	
hello_world_curl_1	curl -o result.log nginx	Exit 0	
hello_world_nginx_1	/docker-entrypoint.sh nginx ...	Up	0.0.0.0:8888->80/tcp

Основные команды: down

\$ docker-compose down

docker-compose down = docker stop + docker rm (для каждого сервиса)

Останавливает все запущенные сервисы, удаляет контейнеры, сети, вольюмы и образы, созданные командой up.

```
→ hello_world git:(master) x docker-compose down
WARNING: Found orphan containers (hello_world_curl_action_1)
mand with the --remove-orphans flag to clean it up.
Removing hello_world_curl_1 ... done
Removing hello_world_nginx_1 ... done
Removing hello_world_app_1 ... done
Removing network hello_world_default
```

Основные команды: build

\$ docker-compose build

Собирает образы всех сервисов проекта

```
→ hello_world git:(master) x docker-compose build
nginx uses an image, skipping
curl uses an image, skipping
Building app
Step 1/7 : FROM ruby:2.7-buster
----> 4ab308e18398
Step 2/7 : RUN apt-get update
----> Running in 64d4f3382fd3
```

Основные команды: run

```
$ docker-compose run --rm [service] [command]
```

Запуск команды в контейнере. Аргумент `rm` по аналогии с `docker` удаляет контейнер после остановки

```
→ hello_world git:(master) x docker-compose run --rm app bash
Creating network "hello_world_default" with the default driver
WARNING: Found orphan containers (hello_world_curl_action_1) for this p
mand with the --remove-orphans flag to clean it up.
Creating hello_world_app_run ... done
root@e1c1dc3de895:/app#
```


Основные команды: logs

```
$ docker-compose logs -f [service]
```

Просмотр логов контейнера. -f выводит логи после запуска команды до остановки работы logs или контейнера.

```
→ hello_world git:(master) x docker-compose logs nginx
Attaching to hello_world_nginx_1
nginx_1 | /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
nginx_1 | /docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
nginx_1 | /docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
nginx_1 | 10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
nginx_1 | 10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
nginx_1 | /docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
nginx_1 | /docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
nginx_1 | /docker-entrypoint.sh: Configuration complete; ready for start up
```

Всем спасибо!

Лига –
лучший
старт
карьеры!

Мы в Лиге!

Умножай
знания –
верь в мечту!

Каждый
день – новый
челлендж!

